ReasonLink: P1 Infrastructure and Baselines

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
P1 Milestone | Due October 19, 2026, 11:59 PM ET

Repository: https://github.com/AaronCherian404/ReasonLink Raw results are in `results/p1/`, the full log is `PROGRESS.md`, and Appendix A lists every step.

I. Summary

Because a VCU118 was not available for Chipyard, we moved from Chipyard and Gemmini to ESP [1], which integrates NVDLA [2] alongside an Ariane core [3]. A VCU118 will be available later through ESP. The neural accelerator is now NVDLA *nv_small* (int8); the symbolic accelerator is still SAT-Accel [4]. The research question, workload, baselines, and ReasonLink mechanisms are unchanged.

For P1, every part of the visual-Sudoku pipeline that does not need the board runs and is checked against a reference, and both baselines run end to end with cycle counters (Table I). The one stand-in is the solver: SAT-Accel cannot run yet, so MiniSat [5] solves SAT-Accel's exact memory image.

TABLE I
Component status.

Component	Status
ESP flow on the server	Ready (ESP 2026.1.0)
NVDLA classifier (int8 LeNet)	Compiled for NVDLA; outputs emulated
Sudoku workload	1,000 puzzles
SAT-Accel host image (C port)	Byte-exact vs. SAT-Accel
SAT solving	MiniSat stand-in
Serial / double-buffered baselines	Running (x86); RISC-V built

II. What Was Built

ESP platform. A departmental server (RHEL 8.10) already has ESP's toolchain: Vivado 2023.2, ModelSim DE 2023.2, and RISC-V GCC 9.2. ESP is pinned to release 2026.1.0, with its NVDLA software installed.

NVDLA classifier. We use the LeNet [6] that ESP ships for NVDLA. Recompiling it reproduces ESP's loadable byte for byte, which pins down ESP's compiler flags. Our Python int8 emulator reproduces ESP's published NVDLA output exactly and generates the per-cell scores that stand in for NVDLA's output buffer. ESP's weights do not match NVDLA's input preprocessing, so we retrained the same network and compiled it with ESP's flags.

Workload. Sudoku is encoded with 729 variables and 11,988 base clauses, and predicted givens are added as unit clauses. We generated 1,000 unique-solution puzzles with 24–40 clues each, in the style of visual Sudoku [7], with MNIST test digits in the given cells.

SAT solver. We ported SAT-Accel's host-side conversion (CNF to device memory image) to C. A golden test compiles SAT-Accel's own parser next to the port and compares every byte. SAT-Accel's emulation needs Vitis `v++` and XRT, which the server lacks, so a MiniSat backend decodes the image the way the device's copy-in would and solves it. PySAT [8] provides the reference answers.

Baselines. A host thread runs SAT-Accel's native flow: argmax, image build, copy into device buffers, then a doorbell and completion poll. A device thread decodes and solves. Every phase is timed with `rdtsc` (`rdcycle` on RISC-V). The double-buffered baseline uses two buffer sets with explicit ownership. We run each baseline *faithful* (all 10.5 MB of allocated buffers cleared and copied, as SAT-Accel's host does) and *lean* (only the bytes in use).

III. Results

Table II collects the functional results. The four baseline runs return identical answers that agree with the PySAT reference: the eight differing grids are puzzles that a misread given leaves with several solutions, and both answers are valid.

TABLE II
Functional results (1,000 puzzles).

Metric	Value
MNIST accuracy, int8: ESP weights → retrained	74.5% → 99.20%
Given-cell accuracy	99.26%
Puzzles solved correctly / UNSAT (misread givens)	79.0% / 19.2%
SAT/UNSAT agreement with PySAT reference	1000/1000
Identical grids (+ valid alternative solutions)	992 (+8)
SAT-Accel image vs. SAT-Accel's own code	13/13 byte-exact
Solver work per puzzle (decisions / conflicts)	1.2 / 0.16
Neural output per puzzle (int8 logits)	1.3 KB
SAT-Accel image in use / buffers migrated	632 KB / 10.5 MB

TABLE III
Baseline breakdown, median μs per puzzle (host x86). Share: $T_{\text{HANDOFF}}/(T_{\text{HANDOFF}} + T_{\text{SAT}})$ for serial runs; exposed fraction of wall time for double-buffered runs.

Run	T_{conv}	T_{xfer}	T_{sat}	Share	Wall
Serial, faithful	1112	880	728	73.6%	2809
Serial, lean	696	13	726	50.5%	1450
Double, faithful	1461	1159	764	70.5%	2703
Double, lean	711	16	743	5.1%	801

Table III gives the first handoff breakdown, and it supports H1 and H2.

- *H1.* In SAT-Accel's native flow the handoff is 74% of end-to-end time (51% when lean). The argmax that actually consumes the neural output takes under 1 μs; the remainder is rebuilding and copying an image of the unchanging base formula: 10.5 MB migrated per puzzle to deliver 36 givens.
- *H2.* Double buffering hides the handoff only when it is shorter than the solve. The faithful handoff (2.6 ms vs. a 0.76 ms solve) stays 70% exposed. The lean handoff is almost fully hidden but still moves 632 KB per puzzle.

T_{sync} is below 1 μs with busy polling, whereas SAT-Accel's host polls once per second.

Limitations and open issues. These are host x86 times. On ESP, conversion runs on Ariane and transfers use DMA, so the numbers will change; the RISC-V build of the same harness is ready for that. T_{sat} is MiniSat, not SAT-Accel; running SAT-Accel is the main open issue. NVDLA's outputs are emulated until NVDLA runs on ESP. Sudoku at these clue counts needs almost no search, so testing H3 will need harder instances. Smaller tooling issues (compiler flags, line endings, a missing `zlib.h` on the server) are resolved and logged in `PROGRESS.md`.

IV. Next Steps

By P2 (Nov 2): run SAT-Accel functionally, either with its HLS kernels as threads or with XRT on a machine that has it; bring up an ESP SoC with Ariane and NVDLA in simulation running the RISC-V baselines; and build the first ReasonLink prototype (literal stream with an on-chip FIFO), measured against Table III. After P2: delta transfers and overlap, SAT-Accel on ESP, and VCU118 runs.

Appendix A
Reproduction Steps

Every step taken for P1. Commands run from the repository root. Parts A and C need a machine with the ESP platform (our server); part B runs on any Linux or WSL machine.

A. ESP server, one-time setup: inventory; ESP pinned with only the NVDLA software; this repository (the server receives code through git, so run `git pull` before each server step).

```
bash env/server_probe.sh
git clone https://github.com/sld-columbia/esp.git ~/esp
cd ~/esp && git checkout 2026.1.0
git submodule update --init \
    accelerators/third-party/NV_NVDLA/sw
git clone https://github.com/AaronCherian404/\
ReasonLink.git ~/ReasonLink
cd ~/ReasonLink
git submodule update --init third_party/minisat
```

B. Local: environment; classifier and workload; SAT-Accel image and solver checks; baselines.

```
bash env/setup_wsl.sh
source ~/venvs/reasonlink/bin/activate
bash workload/fetch_esp_lenet.sh
python workload/nvdla_lenet.py --model esp --check
python workload/train_lenet.py
python workload/dataset.py --num 1000 --seed 0
python workload/reference_solve.py
(cd handoff && make && make golden && make solvecheck)
(cd baselines && make run)
```

C. ESP server, builds. The script recompiles ESP's demo network and checks it against ESP's shipped loadable (md5 `9831533f`). It then compiles ours to `build/nvdla/rl_lenet.nvdla` (md5 `eb4c0813`). `make riscv` builds the static RISC-V baselines.

```
ESP_ROOT=~/esp bash workload/nvdla_compile.sh
(cd baselines && make riscv)
```

The script's compiler call is equivalent to:

```
NVC=~/esp/accelerators/third-party/NV_NVDLA/\
sw/prebuilt/x86-ubuntu
LD_LIBRARY_PATH=$NVC $NVC/nvdla_compiler \
  --prototxt lenet_mnist.prototxt \
  --caffemodel rl_lenet.caffemodel \
  --calibtable rl_lenet.json \
  --cprecision int8 --configtarget nv_small \
  --quantizationMode per-filter --informat nchw
```

Only per-filter quantization with `nchw` input reproduces ESP's loadable; we tried per-kernel with each input format and per-filter with `nchw` and `nhwc`. Inputs are passed by bare file name, because `--calibtable` fails with an absolute path. For the RISC-V build, `make riscv` supplies a declarations-only `zlib.h`, which MiniSat's headers include but the server's toolchain lacks.

References

[1] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with open ESP," in *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, 2020. [Online]. Available: https://www.esp.cs.columbia.edu

[2] NVIDIA, "NVDLA: NVIDIA deep learning accelerator," http://nvdla.org, 2017.

[3] D. Giri, K.-L. Chiu, G. Eichler, P. Mantovani, N. Chandramoorthy, and L. P. Carloni, "Ariane + NVDLA: Seamless third-party IP integration with ESP," in *Fourth Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2020.

[4] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[5] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT)*, ser. LNCS, vol. 2919, 2003, pp. 502–518.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[7] P.-W. Wang, P. L. Donti, B. Wilder, and J. Z. Kolter, "SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[8] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python toolkit for prototyping with SAT oracles," in *Theory and Applications of Satisfiability Testing (SAT)*, 2018, pp. 428–437.